

**MA2006B Uso de álgebras modernas
para seguridad y criptografía Gpo(601)**

Gestor de Identidades 1.0 Casa Monarca

Integrantes del equipo 2

Nombre	Matrícula
Daniela Ortiz Blanco	A00840140
Fernanda Ojeda Tamayo	A01286528
Axel Gabriel Rocha Pérez	A01067854
Ian Roberto Rendón Berlanga	A01384161
Edgar Antonio Larios Rodríguez	A01722806

Profesores

Nombre

Alberto F. Martínez

Daniel Otero Fadul

Raúl Gómez Muñoz

Pedro Leonel Olaya Trejos

Socio Formador

Casa Monarca

Instituto Tecnológico y de Estudios Superiores de Monterrey
Escuela de Ingeniería y Ciencias
Ingeniería en Ciencia de Datos y Matemáticas

11 de Junio del 2026

Índice de Contenidos

Abstract	IV
1. Introducción	1
2. Marco de referencia	2
2.1. Hashing de contraseñas: Argon2id	2
2.2. Fortaleza de contraseñas: entropía de Shannon	2
2.3. Cifrado simétrico autenticado: Fernet	2
2.4. Ciclo de vida de llaves: NIST SP 800-57	2
2.5. Infraestructura de clave pública X.509 y tamaños de clave	3
2.6. Autenticación sin contraseña: WebAuthn / FIDO2 (passkeys)	3
2.7. Bibliotecas y referencias tecnológicas	3
3. Desarrollo	4
3.1. Propuesta de solución	4
3.2. Desarrollo de componentes de Back-End	5
3.3. Desarrollo de componentes de Front-End	5
3.4. Conexión de los componentes	5
4. Resultados	7
4.1. Resultados del Back-End	7
4.2. Resultados del Front-End	7
4.3. Resultados de la solución completa	7
4.4. Casos de uso y tabla de validaciones	7
5. Conclusiones y trabajo futuro	9
Referencias	10
Anexos	11
Anexo A. Comandos principales del proyecto	11
Anexo B. Cuentas de prueba incluidas	11

Índice de Tablas

2.1. Fortaleza de clave en RSA, comparada con algoritmos de clave privada (tomado de Barker et al., 2019).	3
4.1. Casos de uso y validaciones del Gestor de Identidades (elaboración propia).	8
5.1. Cuentas de prueba por rol (entorno de desarrollo).	11

Índice de Figuras

3.1. Arquitectura general de la solución (elaboración propia). 4

3.2. Flujo de canalización de expedientes por niveles operativos (elaboración propia). 6

Abstract

Este reporte documenta el reto *Gestor de Identidades* desarrollado para el socio formador Casa Monarca. El objetivo fue construir el núcleo de identidad y control de acceso de una aplicación web encargada de gestionar expedientes (encuestas) de personas migrantes atendidas por la organización, garantizando confidencialidad, integridad y trazabilidad de datos personales altamente sensibles. La solución entregada es una aplicación Flask con persistencia en SQLite que implementa autenticación por contraseña con hashing Argon2id, política de contraseñas con verificación de entropía y contra filtraciones (HIBP), bloqueo de cuenta por intentos fallidos, protección CSRF, control de acceso basado en cuatro roles (usuario, operativo, coordinador, administrador), cifrado de campo con Fernet (AES-128-CBC + HMAC-SHA256) con inventario y rotación de llaves, una infraestructura de clave pública (PKI) X.509 con verificación desafío–respuesta para acciones críticas, autenticación con passkeys (WebAuthn/FIDO2) y una bitácora de auditoría. Adicionalmente se implementó un flujo multinivel de derechos ARCO. La verificación se realizó mediante una suite de pruebas automatizadas con `pytest` que cubre seguridad de contraseñas, auditoría de cifrado, canalización de expedientes, integración ARCO y vinculación/revocación de passkeys y certificados. El resultado es un gestor de identidades funcional con una base de seguridad alineada a buenas prácticas (NIST SP 800-57, WebAuthn) y listo para evolucionar hacia producción.

1. Introducción

Casa Monarca, Ayuda Humanitaria al Migrante, A.B.P. es una organización de la sociedad civil que acompaña a personas migrantes y solicitantes de refugio en la zona metropolitana de Monterrey. Como parte de su operación, el personal levanta y procesa expedientes con información profundamente sensible: identidad, situación migratoria, condición de vulnerabilidad y datos de contacto. Esta información circula entre distintas áreas operativas, lo que plantea un reto clásico de *gestión de identidades y accesos*: asegurar que cada persona del equipo pueda hacer solo lo que le corresponde, que toda acción quede registrada y que los datos estén protegidos tanto en reposo como durante su manejo.

El reto abordado consiste en diseñar e implementar el **Gestor de Identidades** de la plataforma: el subsistema responsable de autenticar usuarios, autorizar acciones según su rol, proteger criptográficamente los datos de los expedientes y mantener una bitácora auditable. En otras palabras, este reporte responde a la pregunta de qué construimos para que Casa Monarca pueda confiar en *quién* entra al sistema, en *qué* puede hacer cada quien y en que los datos personales que custodia estén cifrados y rastreables.

La plataforma se organiza en torno a cuatro niveles operativos que reflejan el flujo real de trabajo de la organización: el **usuario** captura el expediente, el **operativo** lo revisa, el **coordinador** lo valida y el **administrador** lo cierra y administra el sistema. Cada expediente recorre los estados *borrador* → *en revisión* → *validado* → *cerrado*, y solo los roles críticos (administrador y coordinador) deben demostrar una autenticación reforzada mediante certificados X.509 o passkeys para ejecutar acciones sensibles. La Figura 3.2 resume este flujo de canalización.

A lo largo del documento se describe el marco teórico que sustenta las decisiones criptográficas, la arquitectura y los componentes de *back-end* y *front-end* desarrollados, los resultados obtenidos y su validación mediante pruebas automatizadas, y finalmente las conclusiones y el trabajo futuro previsto.

2. Marco de referencia

Un gestor de identidades descansa por completo en la criptografía. Conviene partir de una definición:

“La Criptografía es la rama inicial de las Matemáticas y en la actualidad también de la Informática y la Telemática, que hace uso de métodos y técnicas con el objeto principal de cifrar, y por tanto proteger, un mensaje o archivo por medio de un algoritmo, usando una o más claves” (Aguirre, 2006).

A partir de esa definición, el diseño del Gestor de Identidades combina primitivas y estándares ya establecidos en la literatura, evitando “criptografía casera”. A continuación se resumen las soluciones existentes revisadas y cómo se aplican.

2.1 Hashing de contraseñas: Argon2id

Las contraseñas nunca se almacenan en claro. Se utiliza **Argon2id**, la función ganadora de la *Password Hashing Competition*, que combina resistencia a ataques de canal lateral (variante *i*) y a hardware dedicado/GPU (variante *d*) mediante un costo de memoria configurable. La implementación se apoya en la biblioteca `argon2-cffi`.

2.2 Fortaleza de contraseñas: entropía de Shannon

Para medir la robustez de una contraseña propuesta se recurre al concepto de entropía.

Una de esas herramientas es la que Claude E. Shannon publicó en el artículo “A Mathematical Theory of Communication”, donde él establece las bases de la llamada “Teoría de la Información” (Shannon, 2001). Dicha herramienta es el cálculo de la Entropía, que en términos generales mide la incertidumbre (aleatoriedad) de la información que se analiza.

La política de contraseñas exige una longitud mínima, un mínimo de entropía estimada y una verificación contra el servicio *Have I Been Pwned* (HIBP) para rechazar contraseñas filtradas.

2.3 Cifrado simétrico autenticado: Fernet

Los datos sensibles de cada expediente se cifran en reposo con **Fernet**, un esquema simétrico autenticado basado en **AES-128 en modo CBC** para la confidencialidad y **HMAC-SHA256** para la integridad/autenticidad del mensaje. Al ser simétrico, utiliza la misma llave para cifrar y descifrar; el sistema mantiene una llave *activa* para nuevos cifrados y conserva llaves *legadas* para poder descifrar durante una rotación.

2.4 Ciclo de vida de llaves: NIST SP 800-57

La gestión de llaves sigue las recomendaciones de **NIST SP 800-57**, complementadas con ISO/IEC 11770 e ISO/IEC 27001. Se define un ciclo de estados `generated` → `active` → `retiring` → `retired` →

destroyed, con un periodo criptográfico (cryptoperiod) sugerido de 90 días y disparadores de rotación por tiempo, volumen o incidente.

2.5 Infraestructura de clave pública X.509 y tamaños de clave

Para reforzar la autenticación de los roles críticos se emplea una **PKI X.509** con una Autoridad Certificadora (CA) local que emite certificados de usuario y verifica acciones mediante un esquema **desafío–respuesta** (la clave privada firma un reto efimero). El dimensionamiento de las claves asimétricas se guía por las equivalencias de fortaleza de NIST (Barker et al., 2019), resumidas en la Tabla 2.1.

Tabla 2.1: Fortaleza de clave en RSA, comparada con algoritmos de clave privada (tomado de Barker et al., 2019).

Módulo clave pública	Tamaño clave privada
2048	112
3072	128
4096	152
6144	176
8192	200

2.6 Autenticación sin contraseña: WebAuthn / FIDO2 (passkeys)

Como segundo factor moderno y resistente a *phishing* se integran **passkeys** bajo el estándar **WebAuthn/FIDO2**. La clave privada nunca abandona el autenticador del usuario; el servidor verifica una firma sobre un reto, ligada al *Relying Party ID* y al origen, lo que impide su reutilización en sitios falsos.

2.7 Bibliotecas y referencias tecnológicas

La implementación se apoya en software ampliamente revisado: el framework web **Flask**; la biblioteca **cryptography** de Python para Fernet, X.509 y operaciones asimétricas (en lugar de, por ejemplo, PyCryptodome); **argon2-cffi** para el hashing de contraseñas; y la biblioteca **webauthn** para passkeys. Estas elecciones priorizan implementaciones mantenidas y auditadas por la comunidad sobre desarrollos propios.

3. Desarrollo

Esta sección responde a la pregunta *¿qué hicimos?*: la propuesta de solución, los componentes de *back-end* y *front-end*, y la forma en que se conectan entre sí.

3.1 Propuesta de solución

Se propuso una **aplicación web monolítica** construida con Flask, con la lógica de negocio, el esquema de base de datos y los ayudantes criptográficos concentrados en un solo módulo (`app.py`), y persistencia en **SQLite**. Las decisiones de diseño más relevantes y su justificación fueron:

- **Monolito Flask + SQLite en lugar de una nube gestionada (p. ej. AWS).** El objetivo es entregar un núcleo de identidad funcional, reproducible y auditable por el equipo docente sin depender de costos ni credenciales de un proveedor externo. SQLite elimina la necesidad de administrar un servidor de base de datos y permite que toda la solución se levante con un solo comando, lo cual es idóneo para una organización pequeña como Casa Monarca. La arquitectura deja la puerta abierta a migrar a PostgreSQL y a un KMS/HSM gestionado en producción.
- **Cifrado a nivel de campo en lugar de cifrado de disco completo.** Solo el contenido sensible del expediente (`encuestas.datos`) se cifra con Fernet, lo que protege los datos incluso ante una copia directa del archivo `database.db` y permite una rotación de llaves granular.
- **Autenticación por capas.** Toda cuenta usa contraseña con Argon2id; los roles críticos (administrador y coordinador) se obligan adicionalmente a configurar certificado X.509 o passkey antes de poder operar, aplicando el principio de defensa en profundidad.
- **Evolución de esquema no destructiva.** Las migraciones se realizan con un patrón idempotente (`ensure_column()`) que agrega columnas sin romper bases de datos existentes.

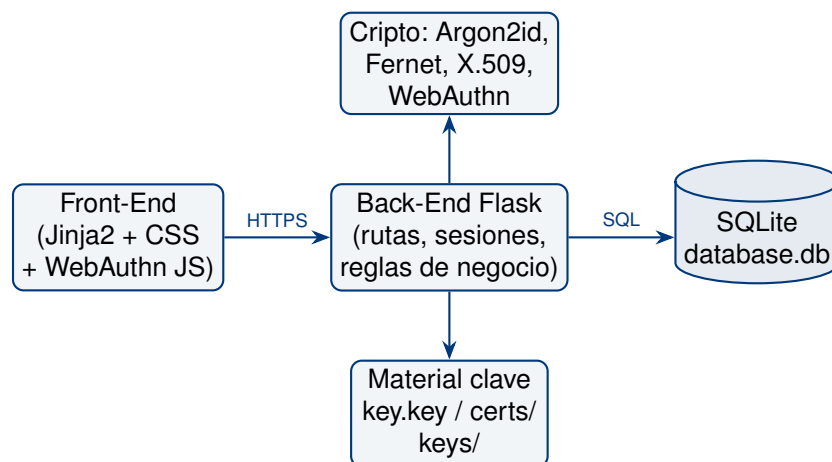


Figura 3.1: Arquitectura general de la solución (elaboración propia).

3.2 Desarrollo de componentes de Back-End

El *back-end* concentra los componentes más relevantes del Gestor de Identidades:

- **Capa de datos.** Acceso a SQLite vía `get_conn()`; creación de tablas en `init_db()` con migración idempotente mediante `ensure_column()`. Tablas clave: usuarios, encuestas (expedientes con datos cifrado), logs (bitácora), certificados, `passkey_credentials`, `encryption_keys`, `encryption_metrics`, `reencrypt_jobs`, `solicitudes_arco` y `login_lockouts`.
- **Identidad y contraseñas.** `hash_password_argon2id()` / `verify_password_argon2id()`; política en `validate_new_password_policy()`, `password_has_minimum_entropy()` y `check_password_pwned()`.
- **Control de acceso.** Cuatro roles en `ROLE_LABELS` / `PERMISSIONS`, verificados con `has_permission()` y `require_role()`; transiciones de estado mediante `next_status_for_role()`, y configuración obligatoria de certificado/passkey vía `enforce_certificate_setup()`.
- **Cifrado de campo y llaves.** `encrypt_data()` / `decrypt_data()` sobre Fernet; carga y anillo de llaves en `_load_key_bytes()` → `_build_keyring()` → `get_cipher_for_fingerprint()`; re-cifrado masivo encolado en `reencrypt_jobs` y ejecutado por `tools/reencrypt_worker.py`.
- **PKI X.509.** CA de desarrollo en `certs/` (`bootstrap_dev_certificates`); emisión/activación/revocación de certificados y verificación desafío–respuesta (`verify_certificate_challenge_response`).
- **Passkeys (WebAuthn).** Registro y login bajo `/auth/passkey/*` y verificación por acción en `/action/passkey/*`, con vinculación a certificados y revocación conjunta.
- **Seguridad transversal.** CSRF global con `csrf_protect()`; banderas de cookies con `enforce_cookie_flags()`; bloqueo de login con `login_lockouts`; y bitácora de auditoría en logs.
- **Derechos ARCO.** Flujo multinivel en `solicitudes_arco` (usuarios → operativos → coordinadores → admin).

3.3 Desarrollo de componentes de Front-End

El *front-end* se construyó con plantillas **Jinja2** y una hoja de estilos única (`static/style.css`), más un módulo JavaScript para passkeys (`static/action-passkey.js`). Las pantallas más relevantes son: `login.html` (acceso y reto de passkey), `dashboard.html`, `survey.html` (captura de expediente), `usuarios.html` y `admin_identidades.html` (gestión de identidades), `admin_pki.html` (certificados y passkeys), `admin_cifrado.html` (inventario de llaves y métricas), `cert_setup.html`, `password_update.html`, `arco.html` (formulario público de derechos ARCO) y `logs.html` (bitácora).

3.4 Conexión de los componentes

El *front-end* y el *back-end* se conectan mediante el sistema de rutas de Flask (40 endpoints, ver Figura 3.1). Cada petición POST pasa por un *hook* `before_request` que valida el token CSRF y aplica las banderas de cookies; la sesión autenticada transporta el rol, que las plantillas consultan para mostrar u

ocultar acciones, y que el back-end vuelve a verificar con `require_role()` antes de ejecutar cualquier operación. Para acciones críticas, el navegador negocia un reto de passkey contra `/action/passkey/options` y envía la firma a `/action/passkey/verify`, que el servidor valida antes de consumir la acción.

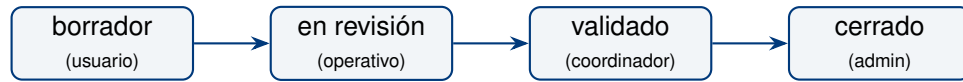


Figura 3.2: Flujo de canalización de expedientes por niveles operativos (elaboración propia).

4. Resultados

4.1 Resultados del Back-End

El back-end resultante expone 40 rutas y un esquema de base de datos completo para el Gestor de Identidades. Quedaron operativos: el alta y autenticación de usuarios con Argon2id y política de contraseñas; el bloqueo de cuenta tras intentos fallidos (5 intentos, ventana de 300 s, bloqueo de 900 s por defecto, configurables); el cifrado/descifrado de expedientes con Fernet, con inventario de llaves, métricas de latencia persistidas en `encryption_metrics` y re-cifrado por lotes; la emisión y verificación de certificados X.509; el registro, login y verificación por acción con `passkeys`; el flujo multinivel ARCO; y la bitácora de auditoría que registra cada acción crítica con usuario, rol y marca de tiempo.

4.2 Resultados del Front-End

Se obtuvo una interfaz en español coherente con los cuatro roles: el menú y las acciones disponibles se adaptan al rol de la sesión; los administradores y coordinadores cuentan con paneles dedicados de gestión de identidades (`/admin/identidades`), PKI (`/admin/pki`) y cifrado (`/admin/cifrado`); y el formulario público de derechos ARCO (`/arco`) permite a cualquier titular ejercer sus derechos sin necesidad de cuenta.

4.3 Resultados de la solución completa

Integrados, los componentes permiten el ciclo completo: un usuario inicia sesión, captura un expediente cuyos datos se cifran en reposo, y lo canaliza a través de los niveles operativos hasta su cierre, exigiendo autenticación reforzada a los roles críticos y dejando rastro auditable de cada paso. La aplicación se levanta localmente con `.venv/bin/python app.py` y sirve en `http://127.0.0.1:5000`; la suite de pruebas se ejecuta con `PYTHONPATH=. .venv/bin/pytest -q`.

4.4 Casos de uso y tabla de validaciones

La Tabla 4.1 resume los casos de uso principales del Gestor de Identidades y su validación. La columna **Evidencias** enlaza a las pruebas automatizadas (archivos bajo `tests/`) que las respaldan.

Tabla 4.1: Casos de uso y validaciones del Gestor de Identidades (elaboración propia).

Caso de uso	Resultado esperado	Evidencias
Seguridad de contraseñas y bloqueo de login	Se rechazan contraseñas débiles/filtradas y se bloquea la cuenta tras intentos fallidos.	tests/test_password_security.py
Auditoría de cifrado	El cifrado/descifrado registra métricas y la auditoría es consistente.	tests/test_encryption_audit.py
Canalización de expedientes	Las transiciones de estado respetan el rol y el orden del flujo.	tests/test_expediente_canalizacion.py
Flujo ARCO multinivel	La solicitud avanza por usuarios, operativos y coordinadores hasta resolverse.	tests/test_arco_integration.py
Vinculación passkey–certificado	Una passkey puede derivar/ligarse a un certificado correctamente.	tests/test_passkey_certificate_linking.py
Revocación de passkeys	Revocar una passkey/certificado invalida la autenticación asociada.	tests/test_passkey_revocation.py
Integración passkey–certificado (endpoints)	Los endpoints de estado y verificación responden de forma consistente.	tests/test_passkey_cert_endpoint_integration.py

5. Conclusiones y trabajo futuro

El reto cumplió su objetivo: Casa Monarca cuenta con un Gestor de Identidades funcional que autentica usuarios con Argon2id, controla el acceso por roles, cifra los datos sensibles de los expedientes en reposo, refuerza las acciones críticas con PKI X.509 y passkey, y mantiene una bitácora auditable, todo verificado mediante pruebas automatizadas. El enfoque de defensa en profundidad y el uso de estándares establecidos (NIST SP 800-57, WebAuthn, Argon2id, Fernet) dan a la solución una base de seguridad sólida para una organización que maneja datos personales especialmente sensibles.

Como trabajo futuro se identifican las siguientes líneas, derivadas del plan de desarrollo del proyecto:

- **Migración completa del firmado de acciones legado a passkeys**, unificando la verificación de acciones críticas.
- **Automatización del ciclo de vida de llaves** (emisión, rotación programada y retiro) conforme a NIST SP 800-57, hoy parcialmente manual.
- **Custodia de material clave en KMS/HSM** (AWS KMS, Azure Key Vault, Google KMS) en lugar del sistema de archivos, para despliegues productivos.
- **Endurecimiento de despliegue**: servir tras un proxy HTTPS, exportar `SECRET_KEY` desde un gestor de secretos y escalar los contadores de bloqueo de login a un almacén compartido (p. ej. Redis).
- **Medición y optimización de latencia** de cifrado/descifrado sobre volúmenes mayores de expedientes, con retroalimentación al usuario.

Referencias

- Aguirre, J. R. (2006). *Libro Electrónico de Seguridad Informática y Criptografía Versión 4.1: Capítulo 1. Presentación del Libro Electrónico.*
- Barker, E., Chen, L., Roginsky, A., Vassilev, A., Davis, R., & Simon, S. (2019). *NIST Special Publication 800-56B Revision 2 — Recommendation for Pair-Wise Key Establishment Using Integer Factorization Cryptography.* NIST Special Publication.
- Barker, E. (s. f.). *NIST Special Publication 800-57 — Recommendation for Key Management.* National Institute of Standards and Technology.
- Shannon, C. E. (2001). A mathematical theory of communication (versión re-impresa). *ACM SIGMOBILE Mobile Computing and Communications Review*, 5(1), 3–55.

Anexos

Anexo A. Comandos principales del proyecto

```
# Instalacion (crea .venv, instala dependencias, genera key.key y certs)
./setup.sh

# Ejecutar la aplicacion (http://127.0.0.1:5000)
.venv/bin/python app.py

# Verificacion de sintaxis
.venv/bin/python -m py_compile app.py

# Ejecutar la suite de pruebas
PYTHONPATH=. .venv/bin/pytest -q

# Regenerar la llave de cifrado Fernet
.venv/bin/python generate_key.py

# Respaldo y restauracion cifrada de la base de datos
.venv/bin/python tools/backup_db.py
.venv/bin/python tools/restore_db.py
```

Listing 5.1: Comandos de instalacion, ejecucion y pruebas.

Anexo B. Cuentas de prueba incluidas

Tabla 5.1: Cuentas de prueba por rol (entorno de desarrollo).

Usuario	Rol	Autenticación reforzada
admin_prod	administrador	Certificado / passkey
admin_cont	administrador	Certificado / passkey
coord_admin	coordinador	Certificado / passkey
operativo_1	operativo	Solo contraseña
usuario_1	usuario	Solo contraseña

Nota: las contraseñas de las cuentas de prueba y todo material clave (key.key, certs/.pem, keys/) son generados en tiempo de ejecución y no deben versionarse ni publicarse.*